pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Initialization and Batch Normalization

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Before and between the steps

Where the weights start

Glorot uniform, Keras's default

He normal, for ReLU

Why not zeros

Vanishing gradients in six sigmoid layers

Between the layers

Batch normalization by hand

Four deep networks, one change each

Moving averages at scoring time

Training mode as a bug

Card file: 14,400 rows fit, 3,600 rows stop or choose, 6,000 validation rows report once.

How an initializer works

1. Random draws

every weight, from a distribution

2. Spread set by fan

fan_in and fan_out of the layer

3. Glorot

tanh, sigmoid, linear (the default)

4. He

twice the variance, for ReLU

You control

Each layer's kernel_initializer; biases start at zero.

What comes out

Starting weights, the same on every run for a seed.

What a layer starts from

Glorot uniform (Keras's default)

```
limit = sqrt(6 / (fan_in + fan_out))
```

19 inputs, 64 units: limit 0.2689. A fresh layer's weights run from -0.2687 to 0.2687; biases start at 0.

He normal (paired with ReLU)

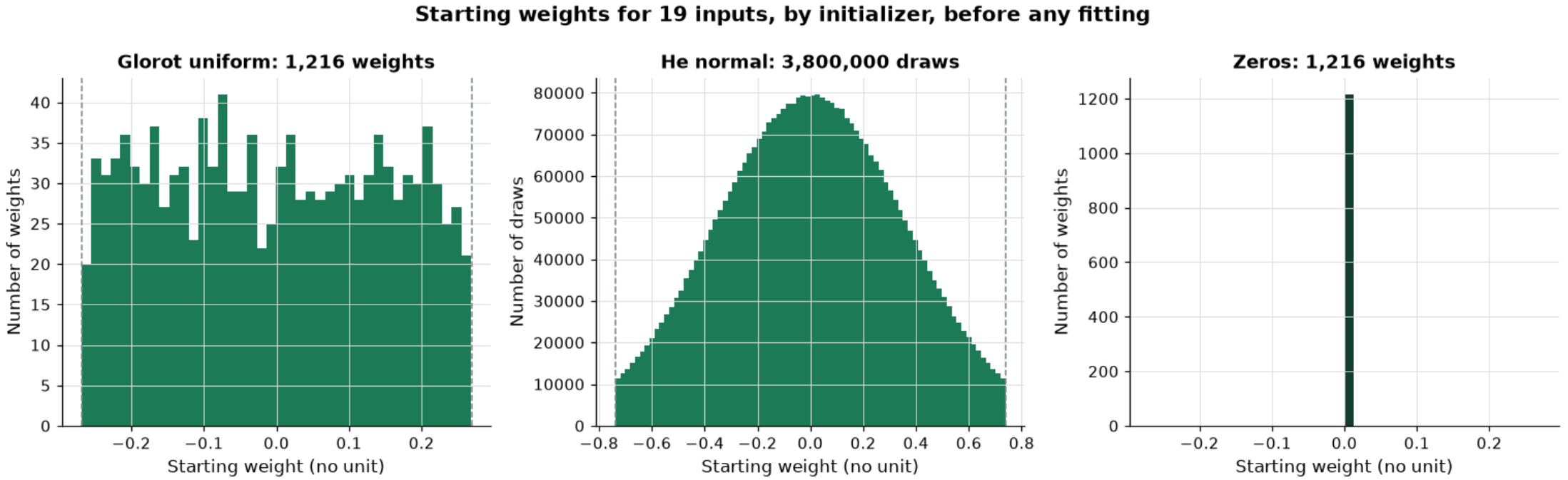
```
standard deviation = sqrt(2 / fan_in)
```

3.8 million draws: 0.32445 against the formula's 0.32444.

Cut at 2 standard deviations after widening by 1 / 0.8796: largest draw 0.7377, 2.27 sd.

Source: keras/src/initializers/random_initializers.py, Keras 3.15.1, read 6 October 2026.

Three starts, drawn



Glorot: every weight inside +/- 0.2689. He: no draw beyond +/- 0.7377. Zeros: one bar at 0.

Nine initializers

Initializer	Aimed sd	Measured sd	Pair it with
zeros	0.0000	0.0000	the biases
random_normal	0.0500	0.0500	small networks
random_uniform	0.0289	0.0288	small networks
glorot_uniform	0.0625	0.0624	tanh, sigmoid, linear; the default
glorot_normal	0.0625	0.0624	tanh, sigmoid, linear
he_normal	0.0884	0.0883	ReLU and its relatives
he_uniform	0.0884	0.0883	ReLU and its relatives
lecun_normal	0.0625	0.0624	SELU
lecun_uniform	0.0625	0.0624	SELU

Drawn for 256 inputs and 256 units. Glorot uses inputs and units; He the inputs only, and aims higher; the measured column agrees with the formulas.

A start of zeros: sixteen copies of one unit

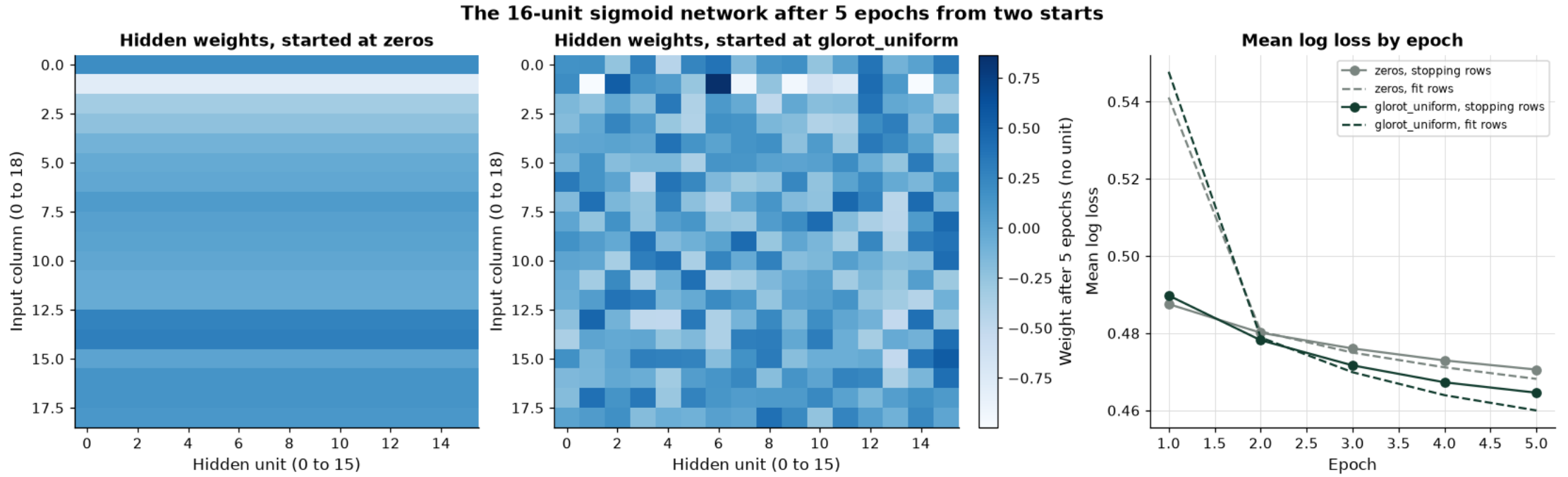
Start	Different hidden units	Stopping loss after 5 epochs
zeros	1	0.4706
Glorot uniform	16	0.4646

Why

Equal weights give equal outputs, equal gradients, equal steps: the units stay equal forever. A random start breaks the tie; the seed makes it the same on every run.

16 sigmoid units, Adam at 1e-3, batch 64, 5 epochs, on the fit rows; loss on the stopping rows.

Sixteen copies of one unit, drawn



Started at zeros, every row is one colour across: 16 equal units. The loss alone does not show it.

How vanishing gradients arise

1. Layer by layer

outputs and gradients pass through weights

2. Too small

they shrink toward zero

3. Too large

they grow or saturate

4. A matched start

keeps them one size

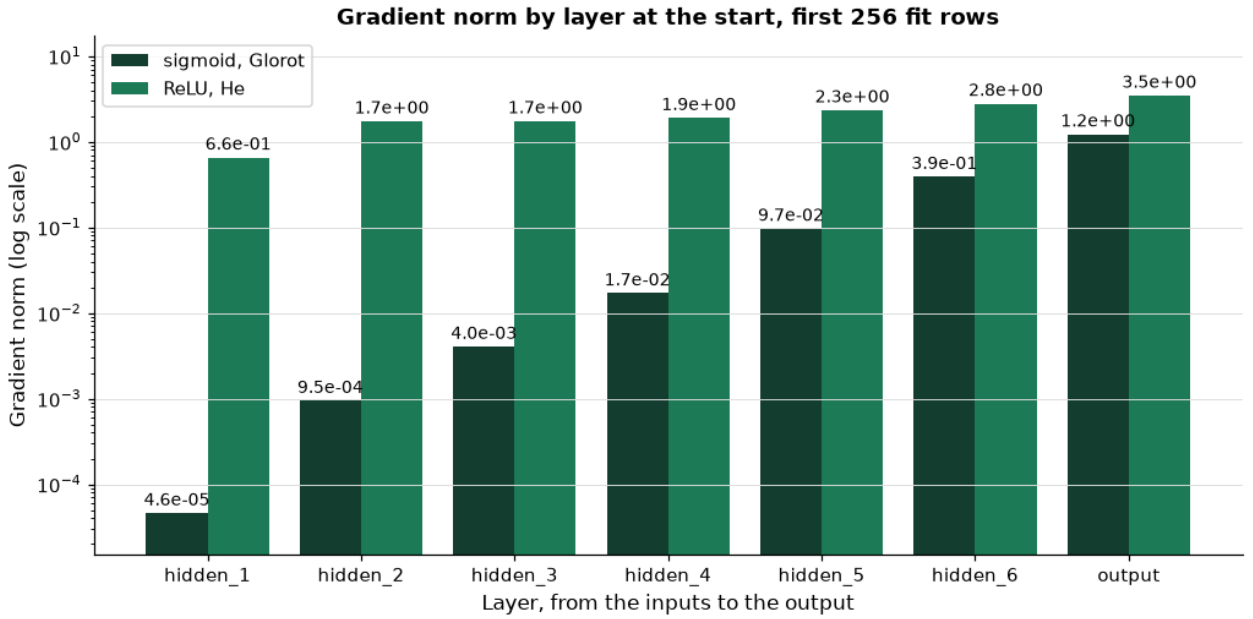
You control

The activation and the initializer.

What comes out

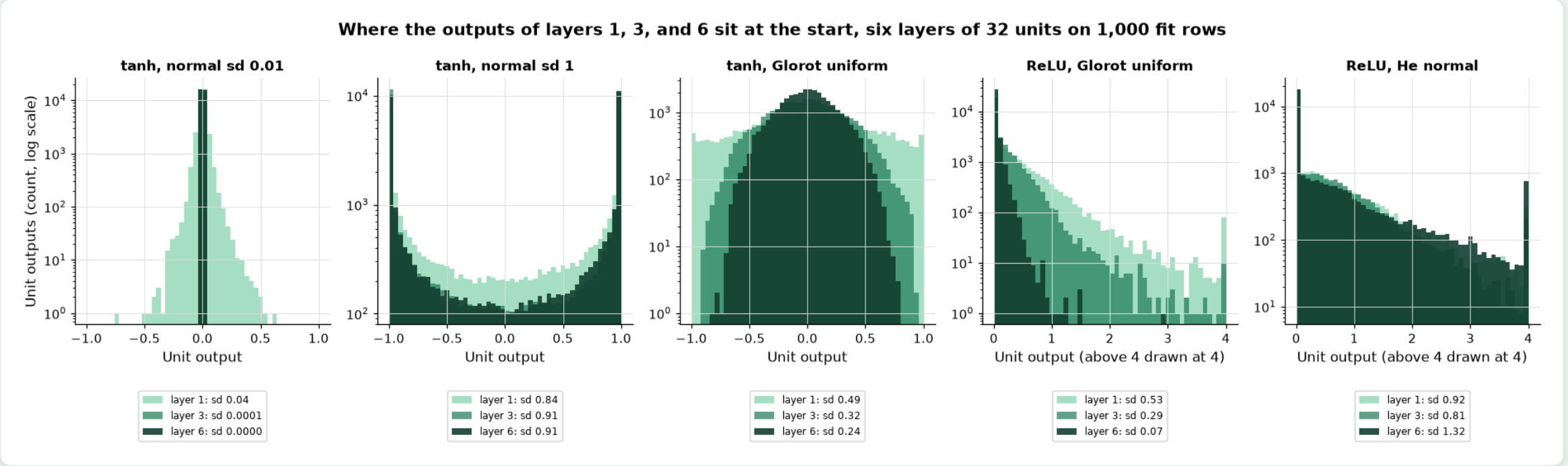
The gradient norm by layer, and the outputs by layer.

Vanishing gradients in six sigmoid layers



Output over first layer: sigmoid 26,544 times (1.22e+00 against 4.60e-05); ReLU with He 5.3 times.

Where the outputs sit



Too small: every unit says zero by layer 6. Too large: tanh saturates at plus and minus 1. ReLU needs He's extra factor of 2.

How batch normalization works

1. Per unit

subtract the batch mean

2. Divide

by the batch's standard deviation

3. Gamma, beta

learned scale and shift

4. Next layer

always sees a steady scale

You control

Where it sits, and momentum, epsilon, center, scale.

What comes out

Standardized outputs, two learned numbers and two moving averages per unit.

Batch normalization by hand

```
(x - batch mean) / sqrt(batch population variance + 0.001) * gamma + beta
```

0.4725 to 0.5473

column means before

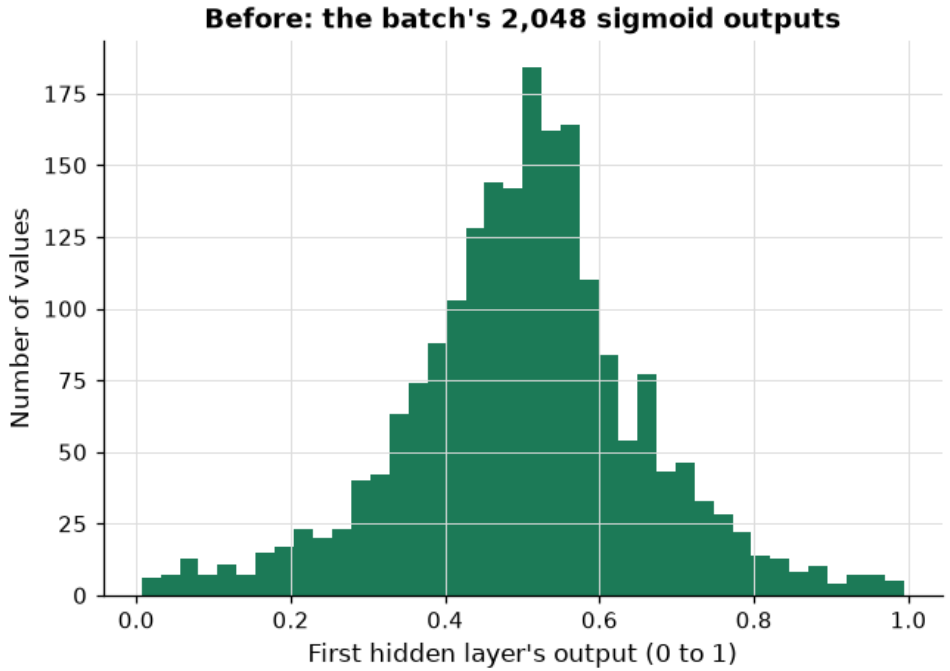
0.9449 to 0.9898

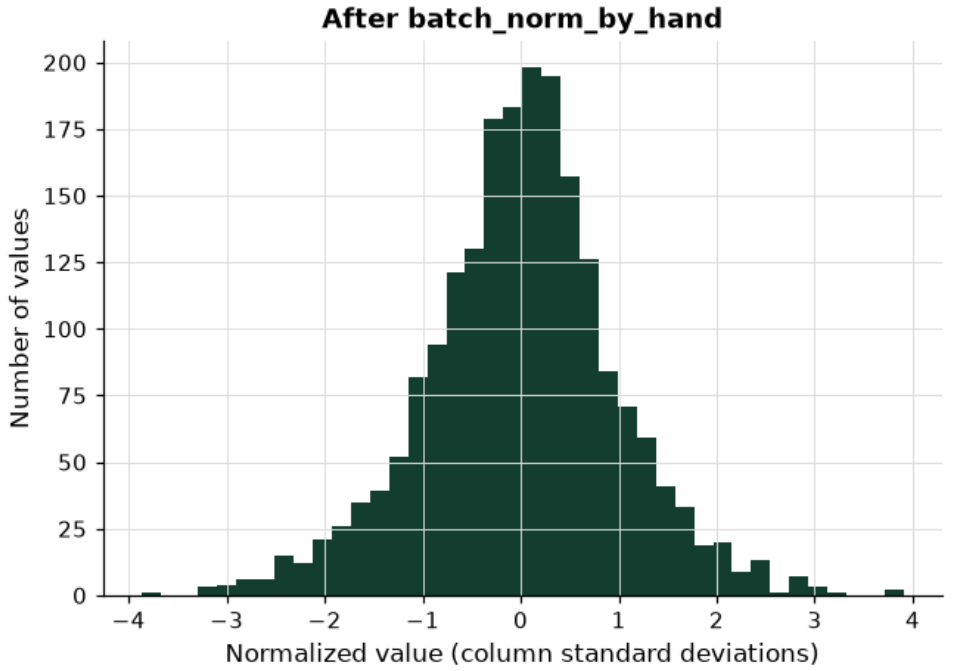
standard deviations after

batch_norm_by_hand equals keras.layers.BatchNormalization in training mode within 1e-5.

A real batch: the first hidden layer's 32 sigmoid outputs for 64 fit rows (standard deviations 0.0912 to 0.2196 before).

Before and after the layer

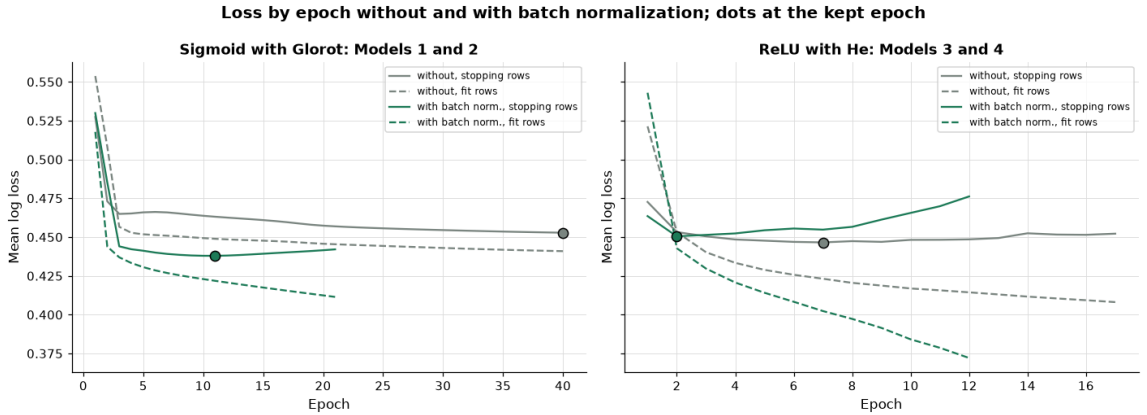




Every column after: mean 0, standard deviation 0.9449 to 0.9898. Training mode: the batch's own mean and variance.

Four deep networks, one setting at a time

Model	Epochs run	Best epoch	Best stopping loss
model_1_sigmoid	40	40	0.4528
model_2_sigmoid_batch_norm	21	11	0.4379
model_3_relu_he	17	7	0.4466
model_4_relu_he_batch_norm	12	2	0.4505



Lowest stopping loss: model_2_sigmoid_batch_norm. Gaps of a few thousandths: day 4's seed spread is the yardstick.

How the layer scores

1. While it fits

uses the batch's mean and variance

2. Every step

moves a moving average toward them

3. At scoring time

uses the moving averages

4. The result

each output depends on its own row

You control

The momentum, and the mode: predict scores, training=True fits.

What comes out

An output that does not depend on the other accounts.

At scoring time: the moving averages

Per unit	Updated by
gamma, beta	the optimizer
moving mean, moving variance	averaging (momentum 0.99)

Model 2

6,721 numbers: 6,337 learned, 384 moving averages.

Scoring mode (predict): account 0 alone 0.2689, among the 6,000 0.2689

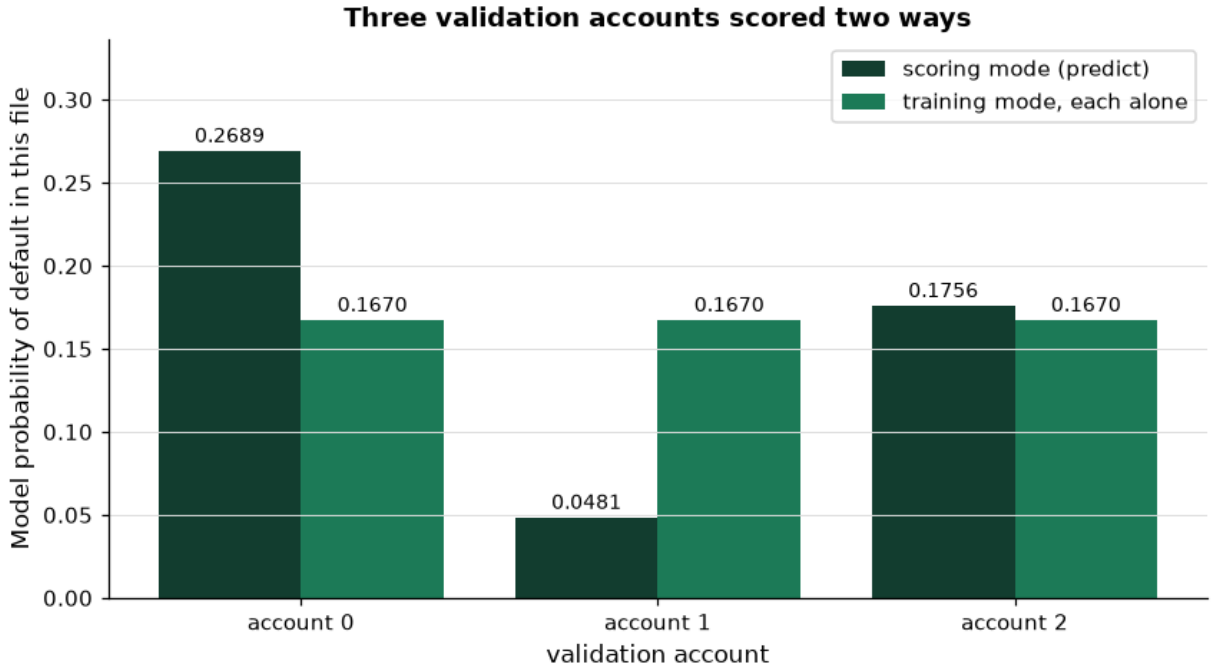
Accounts 0, 1, 2 in scoring mode: 0.2689, 0.0481, 0.1756

Training mode, one at a time: 0.167, 0.167, 0.167. One row: no spread, only beta.

And each training-mode run moved the moving averages: the network changed by being scored.

Score with model.predict, always. training=True only in a cell labelled demonstration.

Two modes, three accounts



Training mode, one row at a time: 0.167 for every account. Scoring mode: 0.2689, 0.0481, 0.1756.

Every setting of BatchNormalization

Setting	Default	What it controls	Tune first
momentum	0.99	memory of the moving averages	short fits only
epsilon	0.001	keeps the division safe	
center	True	learn an offset beta	
scale	True	learn a multiplier gamma	
renorm	False	Batch Renormalization	
synchronized	False	share statistics across devices	

17 settings in all; the notebook's table lists every one. Leave nearly all at their defaults; momentum matters for short fits.

The chosen network, scored once

0.7704

validation ROC AUC

0.5283

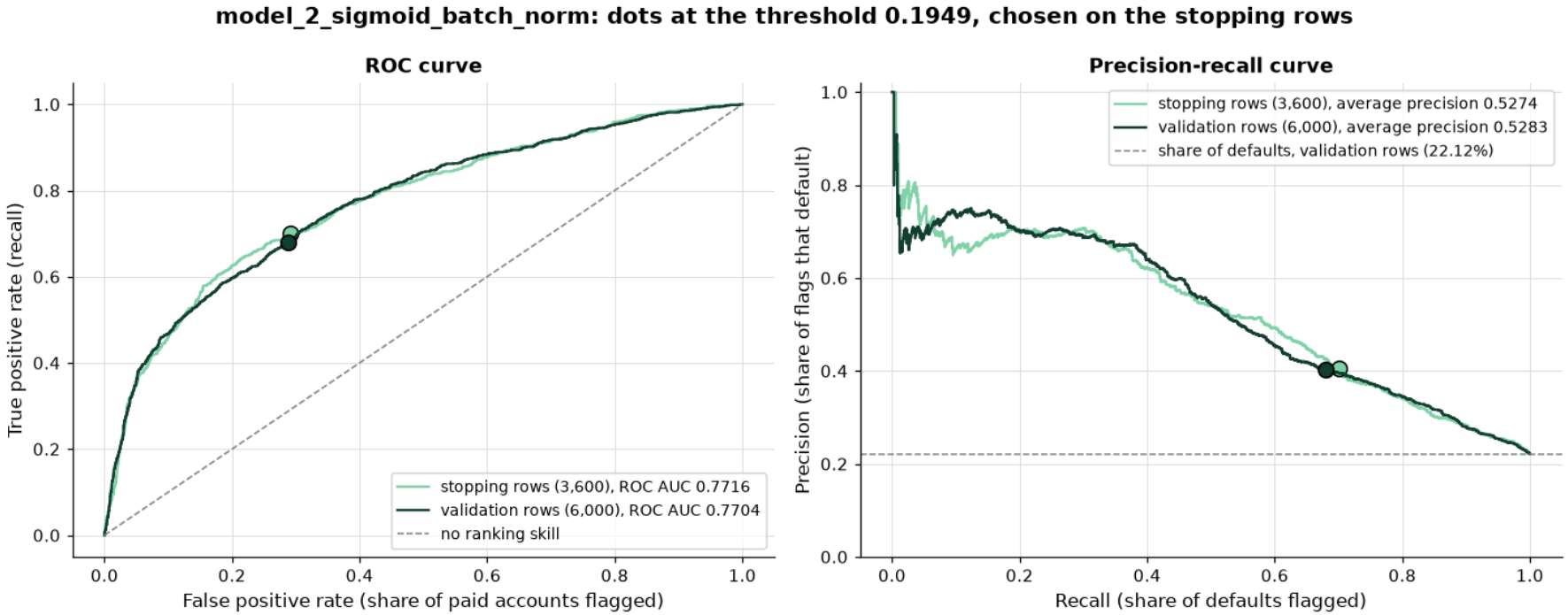
validation average precision

Threshold for recall 0.70, from the stopping rows: 0.1949

On the validation rows: recall 0.6797, precision 0.4020, 2,244 accounts flagged. A threshold carries over only approximately.

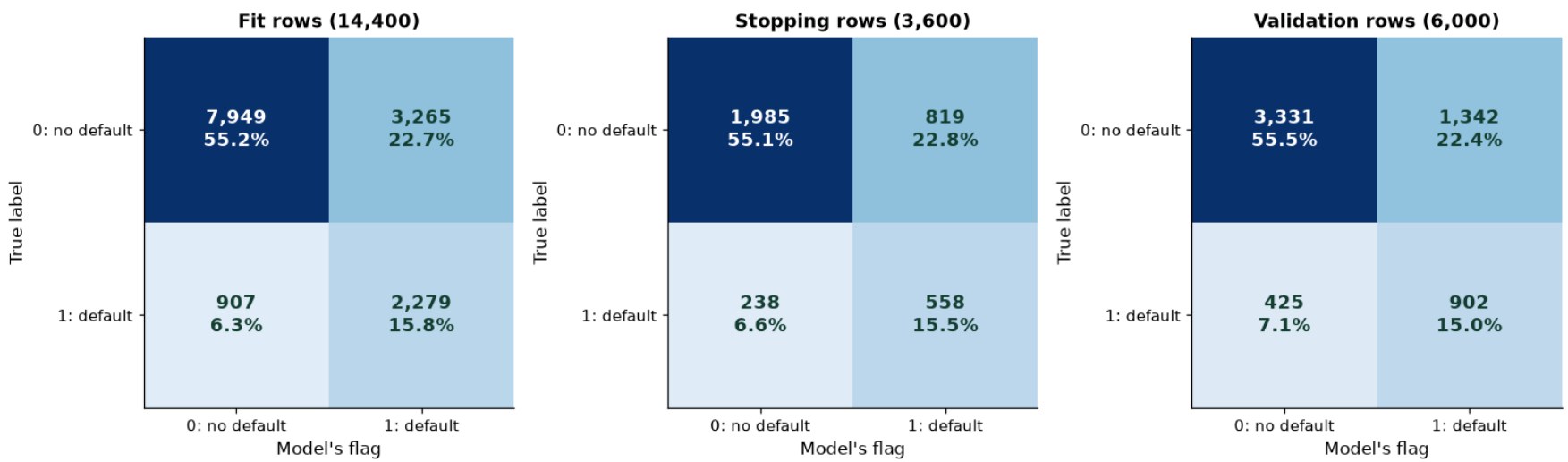
Model probability of default in this file. Labelled validation, never test.

One threshold, two sets of rows



Recall at 0.1949: 0.7010 on the stopping rows, where it was chosen; 0.6797 on validation.

Confusion matrices by split



Rows	Recall	Precision	Accounts flagged
fit rows (14,400)	0.7153	0.4111	5,544
stopping rows (3,600)	0.7010	0.4052	1,377
validation rows (6,000)	0.6797	0.4020	2,244

The name of the layer

Sergey Ioffe and Christian Szegedy, 11 February 2015

"Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift"

arXiv 1502.03167, first version

What the layer did on the card file, you measured in section 8.5.

Source, read 6 October 2026: the arXiv record of the paper, arxiv.org/abs/1502.03167. Bibliographic facts only; the paper itself was not read for this course.

Excel side by side

Excel side by side (tested in Excel)

`=SQRT(6/(19+64))`	Glorot limit 0.268866
`=SQRT(2/19)`	He standard deviation 0.324443
`=(A2-AVERAGE(A$2:A$65))/SQRT(VAR.P(A$2:A$65)+0.001)`	in AH2, across to BM, down to 65
`=AVERAGE(AH2:AH65)`	0 within 1e-12
`=VAR.P(AH2:AH65)`	v / (v + 0.001): just under 1

All 2048 cells equal batch_norm_by_hand within 1e-12 and the Keras layer within 1e-5. VAR.P, not VAR.S: the layer divides by n.

Save the day: git diff --stat HEAD~1

1. Morning

netkit as day 7 left it

2. Code

two functions, four tests

3. Results

results.csv, one row

```
netkit.py      | 45 +++++++++++++++++++++++++++++++++++++++++++++
test_netkit.py | 38 ++++++++++++++++++++++++++++++++++++
2 files changed, 83 insertions(+)
```

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
model_2_sigmoid_batch_norm,validation,0.7704,0.5283,0.1949,0.7055,0.4020,0.6797,0.5052
```

Accuracy beside the all-zero baseline, 0.7788, never to choose.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: model(A, training=True): ROC AUC 0.7659 looks fine, but account 0 is 0.1774 alone and 0.1921 among 6,000.

Code that runs is not the same as code that is right.

What you can do now

- Say what a Dense layer starts from, and what He normal changes
- Explain why a start of zeros fails
- Measure vanishing gradients with gradient_norms
- Do batch normalization by hand, in Python and Excel
- Score in scoring mode: moving averages, never training=True

Week 2, Day 4 Overfitting, dropout, and early stopping